

TOMSEC

CYBERSECURITY HOMELAB / CLASSROOM EDITION

Cybersecurity homelab projects

Self-hosting, hardening and custom proxying

Build private services, harden administration and separate private routing from internet egress. Reproduce the security method with new credentials, then verify access, failure behaviour and recovery.

SELF-HOSTED SERVICES

SECURITY HARDENING

CUSTOM PROXY

EDITION 1.1 / 17 SEPTEMBER 2026

Reported original work is distinguished from new classroom configurations and unperformed deployment tests. No production credentials or private infrastructure export.

Inside the handbook

Start with scope. Build one service at a time. Verify before claiming success.

00 Scope, original work and classroom evidence	3
01 Build the Linux and Docker foundation	5
02 Private remote access with NetBird	8
03 Private HTTPS and the reverse proxy	11
04 A self-hosted password-vault lab	14
05 File sync with Nextcloud, PostgreSQL and Redis	16
06 A separate Immich photo library	19
07 Security monitoring, private logs and DNS filtering	22
08 Harden the host and container boundary	25
09 Build a custom Xray/REALITY privacy proxy	29
10 Separate public applications from private management	36
11 Encrypted backups and a real restore drill	39
12 Publish the learning, not the secrets	42
Validation record	45
Technical sources and provenance	47

Companion repository: [class-project-pack](#). Use the Markdown and script files for copying long commands; PDF line wrapping is for reading. The separate overview is a short introduction for classmates.

00 | Scope, original work and classroom evidence

What these projects demonstrate

The focus is the security engineering behind a self-hosted homelab: reducing unnecessary exposure, separating private administration from app access, protecting credentials, choosing explicit network paths and proving recovery. Installing a container is the start of a project, not its security conclusion.

The original environment used a small Debian host, Docker services and private NetBird connectivity. Some applications were deliberately available remotely through HTTPS. The custom Xray/REALITY setup separated private NetBird traffic from internet traffic using Mullvad egress. Classroom examples recreate those principles without reproducing the original addresses or secrets. [P01]

Original implementation versus teaching controls

Reported original work means the project history describes the setting or outcome. It is not a fresh audit. **Classroom adaptation** means a new example or a safer, smaller reproduction. **Verification task** means the student must run the test and record the actual result; the expected result is not evidence.

Project or control	Basis in original work	Treatment in this edition
Debian/Docker homelab	Multiple self-hosted services reported working	Fresh host with separate app networks and data paths
Private management	Self-hosted NetBird and NetBird-only SSH reported	Own mesh, explicit policies and independent SSH tests
Vaultwarden	Registration/admin disabled; Aegis two-factor setup reported	Fake vault, controlled enrolment, MFA and recovery exercise
Nextcloud	PostgreSQL, Redis, cron, proxy settings and DAV redirects reported	Separate lab with permission and persistence checks
Immich	HTTPS access and phone uploads reported	Independent photo lab; no original photos or library export
Application ingress	Cloudflare Tunnel for selected apps; separate VPS photo edge reported	Private baseline; public-ingress design and deny tests
Monitoring and DNS	Uptime Kuma, Dozzle and AdGuard used in the homelab	Actual service check, private log review and DNS-policy exercise
Custom proxy	Xray/REALITY, Mullvad egress and private NetBird routing reported	Fresh profiles and explicit, limited destination policy
Storage and recovery	Capacity allocations and local backups discussed/used	Encrypted off-source backup and isolated restore drill

The supplied nftables guards, exact SSH snippet, restricted demo proxy policy and automated checks are **new teaching examples**. Do not describe those exact files, their thresholds or their test results as historical production settings. The reference history does not establish a completed end-to-end leak audit, universal kill switch or successful disaster-recovery drill.

Deliberate differences from the original network

The core classroom apps are private by default, reached through an SSH tunnel. This is intentionally narrower than a production system with selected public apps. NetBird control and the custom proxy use separate VPS roles; neither is installed over the other. The proxy begins with an explicit TCP application proxy rather than immediately altering the learner's entire machine with TUN.

A single allowed lab peer and test port replace a broad real overlay route. A dedicated, owned TLS target replaces any identifying transport destination. Public egress has no configured direct-internet fallback. These choices make failures and boundaries easier to test; they are not an anonymity guarantee.

Information excluded from the public pack

Production service domains, host/overlay addresses, account identifiers, credentials, client subscription links, private keys, recovery codes, exact private paths, logs, screenshots, photos and media catalogues are not included. `lab.test` and `example.com` names are placeholders. Generate transport keys, application passwords and provider configuration in your own private workspace.

A proxy, VPN or self-hosted application is not inherently unlawful. The pack teaches authorised deployment and testing only; unsuitable third-party data, unlicensed assets and unauthorised-access material are excluded rather than concealed. No live system is being opened to the class.

Required evidence for each project

Record the aim and permission, component versions, the trust boundary, a working positive test, an attempted denied action, one controlled failure and one limitation. For data services, include a persistence or recovery check. Label unperformed checks **not tested**. Keep full diagnostic output private and share only a reviewed summary or a reconstruction using fictional values.

Start from a fresh VM or spare machine and keep console access before firewall or authentication changes. The following chapters explain where each command runs and how to recover from a failed change. [P01; VALIDATION.md]

01 | Build the Linux and Docker foundation

Original project and learning goal

The original small-form-factor Debian server hosts multiple independent Docker services. The important skill is not the make of computer: it is separating the operating system, application configuration, writable data and recovery copies. This exercise creates a fresh version of that foundation.

For planning, allow roughly 2 CPU cores and 4 GB RAM for a small subset of the examples, or 8 GB for the core exercises with more headroom. These are classroom planning estimates, not measured requirements or a promise of performance. Immich has its own heavier requirements; do not assume it will fit comfortably beside every other service. Start with one project at a time.

Step 1 - Install a separate lab machine

Install a supported Debian release on a disposable VM or spare computer. The reference target is Debian 13. Select an ordinary user account and SSH if remote administration is needed. Use a non-identifying hostname such as `lab-host`. Read every disk-selection screen: installing an operating system can erase the selected disk. Do not practise partitioning on a disk containing the only copy of your files.

After installation, run the following **on the lab server**, not on a Windows PowerShell prompt:

```
sudo apt update
sudo apt upgrade
sudo apt install ca-certificates curl git openssl python3 jq
cat /etc/os-release
free -h
df -h
```

`apt update` refreshes package information; `apt upgrade` applies available updates. `free` shows memory and `df` shows mounted filesystem capacity. Keep full outputs private: device names, usernames and mount paths can identify the real setup. Your public report can record a general capacity and the release.

Step 2 - Install Docker from its supported repository

Use Docker's current Debian instructions, reference S01. On a fresh Debian machine, its repository setup can be performed as follows. Do not use this as an automatic migration script for an already-running container host.

```

sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
. /etc/os-release
printf '%s\n' 'Types: deb' \
  'URIs: https://download.docker.com/linux/debian' \
  "Suites: $VERSION_CODENAME" 'Components: stable' \
  "Architectures: $(dpkg --print-architecture)" \
  'Signed-By: /etc/apt/keyrings/docker.asc' | \
  sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo docker run --rm hello-world
sudo docker compose version

```

The test container should print its success message and exit. The Compose command must exist. These instructions retain `sudo` rather than granting an ordinary account Docker-group access: control of a rootful Docker daemon is highly privileged. Do not treat a Docker-group account as unprivileged. [S01, S02]

Step 3 - Place the teaching repository on the server

Extract `class-project-pack` into your lab user's home folder. When working on a separate laptop, transfer this sanitised folder through your authorised SSH connection. Do not substitute a copy of your original server configuration. The directory containing `README.md` is the repository root.

```

cd ~/class-project-pack
python3 scripts/init_env.py
cd labs/core

```

The helper reads `.env.example`, generates two independent passwords and writes an untracked `.env` with restrictive file permissions. It refuses to overwrite an existing one. An environment file keeps secrets out of committed examples; it is not an encrypted secret store. Root and the Docker administrator can still access service configuration. Do not post `docker inspect` or fully expanded Compose output to a public issue.

Step 4 - Check networking and freeze the images

The illustrative Nextcloud proxy network uses `172.28.250.0/24`. Inspect your existing routes and Docker networks privately before first deployment. When there is an overlap, change `NC_FRONT_SUBNET`, `CADDY_NC_IP` and `NEXTCLOUD_NC_IP` together to an unused private subnet. This is not the author's real network and must not be blindly copied into an existing environment.

```

ip route
sudo docker network ls
sudo docker compose config -q
sudo docker compose config --lock-image-digests -o compose.images.yaml

```

`config -q` checks the model without printing interpolated secrets. The lock command creates an image-digest override. The supplied `lab.sh` automatically uses that override for subsequent commands. Initial image tags are convenient channels, not immutable versions: record the resolved digests in your version record. Do not replace a database major version on an existing data directory without the database's supported migration procedure. [S03]

Step 5 - Start the small core

```
sudo ./lab.sh up -d caddy vaultwarden uptime-kuma
sudo ./lab.sh ps
sudo ./lab.sh logs --tail 30 caddy
```

Logs stay private until reviewed. Vaultwarden starts with registrations disabled. Nextcloud is deliberately started later, after access and HTTPS are understood. All application data lives under the ignored `labs/core/data/` directory. A container can be recreated without losing bind-mounted data, but deleting that directory will remove the actual application state.

The only published web port is `127.0.0.1:8443`. Docker-published ports can interact unexpectedly with host firewall tools, so this lab does not rely only on a reassuring firewall status message. Verify the actual binding and perform a negative reachability test. [S01]

Checkpoint

Docker's test succeeded, the Compose model validates, the selected image digests were recorded, and the three services are running. A browser on another machine cannot open the web apps yet; this is expected until chapters 02-03 establish the authorised path.

Common failures: a missing Compose plugin; lack of disk space; a subnet collision; a port already in use; or using the wrong operating system's package instructions. Diagnose the stated error rather than disabling the firewall or using `chmod -R 777`.

References: S01-S03; original project context P01.

02 | Private remote access with NetBird

Original project and the principle

The original project connected personal devices and servers through a self-hosted NetBird deployment. It allowed remote management without making every administration page a public website. Reproduce the idea in a new network, not by joining the original production account.

Separate three questions: can the device join the network, may it connect to this server and port, and can its user authenticate to the application? A VPN connection does not replace an SSH key or a vault login. [S04, S05]

Step 1 - Prepare a separate controller

This advanced exercise needs a domain you control and a separate publicly reachable Linux VM. At the documentation check for this edition, NetBird's quickstart specifies at least 1 CPU, 2 GB RAM, TCP 80/443 and UDP 3478. Recheck S04 before deployment because the server architecture and required ports can change. Public controller requirements do not mean that home application ports should also be public.

Use an illustrative name such as `netbird.example.com` in your notes, but replace it privately with a real domain you own when installing. A reserved example name cannot obtain a real public certificate. On the controller, install Docker and Compose using the instructions for its own Linux distribution, plus `curl` and `jq`. Establish provider-console access before changing firewall rules.

Step 2 - Download, inspect and run the official installer

On the new controller, create a private working directory outside this publication repository. Use the current bootstrap from S04. Download it to a file instead of piping an unseen response directly into a shell:

```
mkdir -m 700 ~/netbird-bootstrap
cd ~/netbird-bootstrap
curl -fL \
  https://github.com/netbirdio/netbird/releases/latest/download/getting-started.sh \
  -o getting-started.sh
less getting-started.sh
sha256sum getting-started.sh
bash getting-started.sh
```

Record the installer checksum and resulting component versions privately. The bootstrap URL is a moving release endpoint; retaining this information matters for reproducing what you actually installed. Do not commit its generated configuration, databases or identity material to the class repository.

For a new isolated controller, follow the official included reverse-proxy path. Decline optional features that publish private applications to the Internet: this exercise only needs device connectivity. Create the first administrator promptly, keep account recovery material private, and configure appropriate account protection. Generated file names and screens are version-sensitive; use S04 rather than an old screenshot when they differ.

Step 3 - Enrol only your own lab devices

Install the NetBird client using its official platform instructions, S06. On a Linux lab server, a readable installation-script workflow is:

```
curl -fL https://pkgs.netbird.io/install.sh -o /tmp/netbird-install.sh
less /tmp/netbird-install.sh
sudo bash /tmp/netbird-install.sh
sudo netbird up --management-url https://netbird.example.com
sudo netbird status
```

Replace the example management URL with your own before running `netbird up`. Complete the authentication flow shown by the client. Do not publish the login URL, setup key or full status output. Install and enrol the laptop separately, using its official client. Each student should have an independent lab or an explicitly authorised, instructor-managed environment.

Step 4 - Define a small access policy

Create a group for your administration laptop and a different group for your lab server. Add an explicit policy allowing the laptop group to initiate TCP connections to the server group on port 22. For this pack, HTTPS is carried inside an SSH local forward, so you do not need to permit a direct web port.

Check the new policy before removing any broad default policy. An existing all-to-all grant can still allow access despite a second, narrower rule. Once the intended route works, remove the broad grant and test again. Keep a console or known-good session available while changing access control. [S05]

Create a fresh SSH key on the laptop, install only its public key into the lab account, and verify a new key-authenticated session before disabling password logins. Verify the server's host-key fingerprint through your own trusted console rather than accepting an unexpected changed key blindly. The private key must never leave the laptop's secret storage or appear in a repository.

Step 5 - Prove both the allow and the deny

Test	Expected result
Authorised laptop connects to lab server TCP 22	Connection succeeds, then SSH authentication is required
A separate unenrolled or unpermitted test device attempts the same path	Connection is denied or unreachable
Laptop attempts an ungranted lab service port	No connection
NetBird is disconnected on the laptop	Private route is unavailable

Use only your own devices and addresses for these tests. A failed `ping` is not conclusive when ICMP was never allowed; test the exact permitted TCP service. Likewise, a connected client does not prove that the intended access policy is correct. Use application-level tests as well as the client status.

Troubleshooting and limitations

When access fails, work in order: controller reachability, peer enrolment, policy, target address, server listener and SSH authentication. Do not install an additional VPN or broad route before understanding which layer failed.

This chapter explains the fresh bootstrap and access design; it does not ship an immutable copy of the rapidly changing NetBird server stack. Preserve the versions you install and consult its maintenance documentation for upgrades and recovery. The chapter is not a claim that the author's live setup was rebuilt or re-audited during preparation.

References: S04-S06; original context P01.

03 | Private HTTPS and the reverse proxy

The data path

The original work used reverse proxies and different access paths for public and private services. This class version deliberately chooses a smaller access surface: one loopback listener, an authenticated SSH tunnel and local HTTPS.

```
Browser on student laptop
-> https://check.lab.test:8443/healthz
-> laptop loopback / SSH local forward
-> authorised SSH connection to lab server
-> server 127.0.0.1:8443
-> Caddy HTTPS listener
-> selected application or static files

Database and Redis: no host ports published
Other LAN/Internet clients: no direct web listener provided
```

The hostname selects the application, the certificate authenticates that name, the tunnel supplies the authorised network path, and the application still controls its own users. These are different jobs, not interchangeable features.

Step 1 - Start the SSH forward on the laptop

Keep this terminal open. Replace the account and address with your own values. The literal placeholder below is not a working destination.

```
ssh -N -o ExitOnForwardFailure=yes \
-L 127.0.0.1:8443:127.0.0.1:8443 \
labuser@YOUR_LAB_SERVER_ADDRESS
```

The left-hand listener is on the laptop; the right-hand destination is evaluated from the SSH server. A local port forward does not require a router port-forward for 8443. When the SSH process stops, the browser path stops too. Do not change `127.0.0.1` to `0.0.0.0` to share the tunnel with other machines.

When everything runs on one Linux computer, no SSH tunnel is needed. Keep the same localhost names and certificate steps.

Step 2 - Add fictional local names on the laptop

Add the following single line to the laptop's hosts file using an administrator editor. On Linux the file is `/etc/hosts`; on Windows it is `C:\Windows\System32\drivers\etc\hosts`.

```
127.0.0.1 vault.lab.test cloud.lab.test status.lab.test check.lab.test
```

This must be on the computer running the browser, not only on the server. It maps names to the local tunnel entrance. Keep the file's other entries and remove this line after the lab. These names are not public DNS records.

Step 3 - Export only the lab's public CA certificate

Caddy's `tls internal` uses a local certificate authority. Containers cannot automatically make your separate laptop trust that authority. Export its public root certificate from the running server container, not its private key. [S07]

Run **on the server**, from `labs/core`:

```
sudo ./lab.sh cp \
  caddy:/data/caddy/pki/authorities/local/root.crt \
  ./classlab-root.crt
sudo chown "$(id -u):$(id -g)" classlab-root.crt
openssl x509 -in classlab-root.crt -noout -fingerprint -sha256
```

Transfer `classlab-root.crt` to the laptop using your already verified SSH connection. Compare its SHA-256 certificate fingerprint with the value observed through the trusted server session. A certificate downloaded from an unknown page is not a trustworthy basis for adding a new root authority.

For the browser exercise, import it as a trusted website authority in a **dedicated lab browser profile**. In Firefox, this is in certificate management under Privacy & Security, Authorities. Do not override managed college security settings. Only import your own verified lab CA, understand that a trusted root has broad signing power, and remove it when the exercise ends. Some native applications do not trust user-added authorities; do not disable their TLS verification as a workaround.

Step 4 - Test without bypassing certificate verification

From the laptop directory containing the copied certificate:

```
curl --cacert classlab-root.crt \
  --resolve check.lab.test:8443:127.0.0.1 \
  -I https://check.lab.test:8443/healthz
```

The expected result is an HTTPS response for the lab health endpoint, ordinarily 200 OK. This command explicitly supplies the trusted CA and destination; it does not use `-k` or otherwise disable certificate checks. In Windows PowerShell, use `curl.exe` when `curl` is an alias to another command.

Open `https://check.lab.test:8443/healthz` in the lab browser. Then try `https://status.lab.test:8443`. Keep the port suffix: the container's internal 443 is intentionally mapped to host 8443, and no HTTP redirect port is published.

Step 5 - Inspect the proxy behaviour

Open `labs/core/Caddyfile`. Each site has one responsibility. The vault route forwards to Vaultwarden; the cloud route forwards to Nextcloud; the status route forwards to Uptime Kuma; the check route returns a fixed health marker. There is no website, content-management system or site-asset directory. The proxy does not expose database ports or mount the Docker socket.

Nextcloud receives an explicit external HTTPS URL and trusts only the designated Caddy address on its dedicated frontend network. Do not replace this with `trusted_proxies = all addresses`. Correct proxy trust matters for client-IP handling and security features. [S10]

Failure tests and troubleshooting

Close the SSH tunnel: the laptop page should stop responding. Test the lab server's LAN address from another machine: port 8443 should not provide this site. Inspect the Compose binding: it should say `127.0.0.1`, not an unspecified host address. These tests are stronger evidence than assuming a VPN or firewall makes every service private.

A certificate warning normally means the wrong name, missing trust or a different CA. A connection refusal normally means no tunnel or no listener. A `502` points toward the proxy's upstream application. A correct HTML login page is not the same as a functioning mobile API.

The original RSS work exposed this last distinction: an interactive access login can return HTML where an app expects an API response. Service-token support depends on the client being able to send the required headers. Prefer an appropriate private access path over removing authentication from every API route. The RSS/mobile fix is not claimed as completed here. [S16]

References: S07, S10, S16; original context P01.

04 | A self-hosted password-vault lab

Original project and the classroom boundary

The original Vaultwarden deployment was reported working. This classroom version makes closed registration, a disabled server administration interface and account-level two-factor authentication explicit goals. It teaches controlled access and persistence, not hosting real classmates' passwords.

Vaultwarden is a community implementation compatible with Bitwarden clients; it is not the same product as Bitwarden's official server. Its web vault needs a suitable secure browser context, so complete the HTTPS chapter first. [S08]

What the configuration does

In `labs/core/compose.yaml`, the service has a persistent `/data` mount but no published host port. Only Caddy can reach it through its application network. `DOMAIN` points to the actual classroom URL, including `:8443`. `SIGNUPS_ALLOWED` starts false; `INVITATIONS_ALLOWED` is false. No `ADMIN_TOKEN` is configured, so the separate server administration panel is not enabled. These are explicit choices, not evidence that an installation can never be compromised. [S09]

The vault's user account and the server administration panel are separate concepts. A person who can sign in to their vault does not automatically need the server-wide administration interface.

Step 1 - Start and verify the empty service

Run on the lab server from `labs/core`:

```
sudo ./lab.sh up -d vaultwarden caddy
sudo ./lab.sh ps
```

Open `https://vault.lab.test:8443` through the verified laptop tunnel. There should not yet be an account you can use. A real deployment must not be left with open first-user enrolment while other people can reach it.

Step 2 - Create one disposable account deliberately

Use only a fictional lab identity and a new passphrase you do not use anywhere else. While access is restricted to your own machine, privately edit `.env` and set `VW_SIGNUPS_ALLOWED=true`. Recreate the service:

```
sudo ./lab.sh up -d --force-recreate vaultwarden
```

Create your one demonstration account in the web vault. Immediately set `VW_SIGNUPS_ALLOWED=false` again and repeat the same recreate command. A restart alone may not apply changed container environment values; the explicit recreation makes that transition clear.

Use a separate browser session to attempt a second registration. It should be rejected. Merely hiding a registration button would not establish that the registration API itself is disabled. Preserve a sanitised description of the negative test, not a screenshot containing the real password or email.

Step 3 - Create and synchronise a fake vault item

Add an item named `Classroom Demo` using a fictional service such as `https://example.com`. Enter only generated demonstration credentials. Lock and unlock the vault. Then log out and sign in again. Verify that the item persists and that a different, incorrect passphrase does not unlock the account.

A test browser extension is optional and should use a separate lab profile. Point it at the classroom server URL only after checking certificate trust. Do not move your everyday password manager account to this exercise server.

Step 4 - Demonstrate two-factor authentication

Using the application's security settings, enable a supported second factor for the disposable account. Keep the resulting setup QR code, seed and recovery code private. Do not include them in evidence even for a demo: teaching everyone to publish recovery material is a bad habit. Log out and verify that the next login requires the expected second factor.

Account two-factor authentication does not rescue a leaked master password in every possible attack, and it does not replace server maintenance or backups. Document it as one layer of protection, not as a complete security claim.

Step 5 - Verify persistence and recovery requirements

Recreate the container without deleting `data/vaultwarden`:

```
sudo ./lab.sh up -d --force-recreate vaultwarden
```

Log in again and verify the fake item. The exercise demonstrates separation of container lifecycle from persistent state. It is not a backup test until the data is restored into a separate environment, covered in chapter 11.

A safe recovery set includes the application's database and associated persistent files, not just the visible Compose file. The supplied cold-backup approach stops writers before taking an encrypted copy, avoiding a casual live-copy of a changing SQLite database. Keep any encrypted backup's recovery material away from the same disk that it is meant to protect.

Troubleshooting and evidence

For a page that will not load, test the tunnel, name and CA trust before changing Vaultwarden. For a failure to create an account, check the deliberate signup window rather than permanently opening registrations. For a new configuration that appears ignored, check container recreation and whether an existing application configuration overrides an environment option.

Finish with: a trusted HTTPS page, one disposable account, rejected second registration, demonstrated second-factor login, and persistence after recreation. No real secrets should appear in screenshots, terminal recordings or the report.

References: S08-S09; original context P01.

05 | File sync with Nextcloud, PostgreSQL and Redis

Original project and the architecture

The original Nextcloud project provided remote file access with a defined storage budget. Mobile and desktop use was reported in the project history. This teaching version uses PostgreSQL, Redis and a separate, disposable data tree to make the application relationships reproducible.

The image used here is the community-maintained Nextcloud Docker microservice image, not the vendor's All-in-One deployment. We use it to make the application, database, cache and job runner visible as separate components. [S11]

```
Caddy -> Nextcloud web application -> PostgreSQL metadata
      |-----> Redis coordination/cache
      |-----> Managed file directory
Cron -> same configuration and managed files -> scheduled jobs
```

A database record and an uploaded file solve different storage problems. An empty database with an old data folder is not automatically a complete restore. Likewise, Redis is not your recovery copy of uploaded documents.

Step 1 - Understand the supplied variables

Read `x-nc-env` in `compose.yaml`. The database host is the Compose service name `db`, not `localhost`: inside a container, `localhost` refers to that container. The database name and account are dedicated to this application. The generated database password is shared only with the components that need it.

The public-facing classroom name is `cloud.lab.test:8443`. The external protocol and URL overrides tell Nextcloud how clients reached it through Caddy. Only the specific proxy address is trusted.

`REDIS_HOST=redis` selects the separate Redis service. The application and cron job runner use the same persistent directory. These environment options are defined in the image documentation. [S10, S11]

Step 2 - Start the database and application

Run on the lab server, in `labs/core`:

```
sudo ./lab.sh up -d db redis nextcloud
sudo ./lab.sh ps
sudo ./lab.sh exec --user www-data nextcloud php occ status
```

On first start the image needs to initialise the installation. Until that finishes, `occ status` can fail or show an incomplete state. Inspect local logs privately and retry the status check; do not delete the database to make an initialisation warning disappear.

Once installed, the status command should show the application installed and not requiring a database upgrade. Then start the job runner and select cron:

```
sudo ./lab.sh up -d cron
sudo ./lab.sh exec --user www-data nextcloud php occ background:cron
```

This edition keeps the database/cache network private and provides the cron container a separate outbound network for jobs that require Internet access. No database or Redis port is published on the host.

Step 3 - Sign in and create a normal user

Open `https://cloud.lab.test:8443` in the trusted lab browser. The initial admin name is `labadmin`; its generated password is in the server's private `.env`. Read it privately without streaming the terminal to the class. These values are used during initial installation, not as a universal password reset method.

Create a normal account such as `student-demo` and give it a small quota, for example 1 GB. Use the normal account for the file exercise. Keep administration separate from routine use so that you can explain which tasks require elevated application privileges.

Upload three harmless files with distinct content. Rename one through Nextcloud, download another and compare its hash with the original. Check that a file uploaded under one user is not visible to another user without an intentional share. A successful admin login alone does not test user isolation.

Step 4 - Test the client workflow

A desktop sync client can connect to the same classroom URL while the SSH tunnel is open and its certificate is trusted. Use a new local sync folder, not a directory containing the only copy of your coursework. Create one small text file locally, allow it to sync, then modify it through the web interface and observe the client's behaviour.

Distinguish **synchronisation** from **backup**. A synchronised deletion or bad change can propagate. The existence of a second client does not prove that you can recover an earlier server state after a hardware failure.

A phone is an advanced extension because the basic SSH-forwarded lab is tied to the laptop. Do not point a phone at a public login page and assume its native API has passed all tests. Use a separately planned private HTTPS path and verify its authentication, certificate trust and upload behaviour.

Step 5 - Check the operational details

Review the administration overview. Explain each remaining warning rather than claiming a warning-free deployment without checking. Confirm that background jobs run. Test the DAV discovery redirect from the laptop:

```
curl --cacert classlab-root.crt \
  --resolve cloud.lab.test:8443:127.0.0.1 \
  -I https://cloud.lab.test:8443/.well-known/caldav
```

The Caddy configuration should return a redirect to the application's DAV path using the same external HTTPS hostname and port. That verifies a specific integration detail; it does not establish that every calendar client works.

The application's quota is not a hard quota for the entire physical disk. Database growth, previews, versions, trash, logs and backups also need capacity. Keep Immich's media directory separate from Nextcloud's managed data directory. Do not manipulate files underneath Nextcloud as if its metadata will always update automatically.

Troubleshooting, backup and completion

An untrusted-domain message points to hostname configuration. An HTTP redirect or mixed-content problem points to the proxy/external URL. A database connection failure points to credentials, service discovery or database readiness. A stuck sync client may involve permissions, quota, conflict handling or an interrupted tunnel. Reinstalling a desktop client is not a substitute for understanding the server-side error.

A recovery set includes configuration, managed files and the matching database. Use chapter 11's stopped-stack recovery exercise, or follow Nextcloud's documented maintenance-mode/database backup procedure for a production design. [S12]

Finish with: normal-user login, a verified file round-trip, an isolation test, working scheduled jobs, documented quota/headroom and a successful separate restore drill before placing valuable files on the service.

References: S10-S12; original context P01.

06 | A separate Immich photo library

Original project and why it stays separate

The original photo service was reported working through HTTPS, with the initial administrator created and phone media uploads running. Its storage was kept separate from Nextcloud. This class lab preserves that separation and uses only harmless sample photos you own.

Do not treat a photo uploader as proof that the server has a recoverable backup. Immich's database and photo/video files are distinct parts of the recovery set. A database backup alone does not contain the media. [S15]

Step 1 - Check capacity before installing

Read the current installation and requirements pages, S13-S14. At the check for this edition, the quickstart calls for at least 6 GB RAM and two CPU cores. Leave additional headroom for the operating system and any other services. These are not performance benchmarks for the original hardware. On a small machine, stop other labs or use a separate VM rather than launching everything together.

Prepare an empty, sufficiently large local storage location. Do not reuse the Nextcloud data directory or its PostgreSQL database. Do not place the live Immich database on a network share. Record the actual release chosen before starting; application and database dependencies must remain compatible.

Step 2 - Download a matching official release pair

Immich's supporting services and database extensions change over time. This pack intentionally does not invent a replacement Compose stack. Open the project's official release page and select a supported, non-prerelease tag. Use that **same tag** for both required files.

Run on the lab server from the repository root:

```
mkdir -p labs/immich/upstream
cd labs/immich/upstream
read -r -p 'Official release tag, for example vX.Y.Z: ' IMMICH_RELEASE
curl -fL \
  "https://github.com/immich-app/immich/releases/download/$IMMICH_RELEASE/docker-compose.yml" \
  -o docker-compose.yml
curl -fL \
  "https://github.com/immich-app/immich/releases/download/$IMMICH_RELEASE/example.env" \
  -o .env
chmod 600 .env
```

Do this only in a new empty directory. Downloading to `.env` over an existing installation would overwrite its settings. The example `vX.Y.Z` is descriptive, not a real release tag to enter. Confirm both downloads succeeded before going further. Retain their checksums and the chosen release in your private build record. The ignored `upstream/` folder is not part of the public repository.

Step 3 - Configure private data and pin the version

Read the downloaded files. In `.env`, set `UPLOAD_LOCATION` to an empty photo library location and `DB_DATA_LOCATION` to a separate local database location. The relative defaults can be used for a disposable exercise with enough disk space. Set `IMMICH_VERSION` to the exact release tag selected above rather than a moving major-release alias.

Generate an independent database password with `openssl rand -hex 24` and place it privately in `DB_PASSWORD`. Do not record it in screenshots or reuse the Nextcloud password. Keep other database identifiers consistent with the official file. Do not change a password in `.env` later and assume an existing database's account password has automatically changed.

Step 4 - Replace the public port binding before first start

Create `compose.private.yml` beside the downloaded files with:

```
services:
  immich-server:
    ports: !override
      - "127.0.0.1:2283:2283"
```

This requires a Compose version supporting `!override` (2.24.4 or newer). It replaces the upstream port list; an ordinary merged list can accidentally retain the public binding. Verify the model before starting. A changed upstream service name or target port must be reconciled with the selected official release.

```
sudo docker compose -f docker-compose.yml -f compose.private.yml config -q
sudo docker compose -f docker-compose.yml -f compose.private.yml \
  config --format json | python3 ../../scripts/check_bindings.py
sudo docker compose -f docker-compose.yml -f compose.private.yml \
  config --lock-image-digests -o compose.images.yml
sudo docker compose -f docker-compose.yml -f compose.private.yml \
  -f compose.images.yml up -d
```

The helper prints port bindings, not environment secrets. It fails when a published port is not explicitly loopback-bound. Keep using all three `-f` arguments for later commands so the privacy override and image lock remain in force. Do not run a bare `docker compose up` here and drop the overrides.

Step 5 - Open a local test connection

On the laptop, open an independent tunnel:

```
ssh -N -o ExitOnForwardFailure=yes \
  -L 127.0.0.1:2283:127.0.0.1:2283 \
  labuser@YOUR_LAB_SERVER_ADDRESS
```

Open `http://127.0.0.1:2283` on that laptop. This HTTP connection is local to the laptop and then carried through the SSH connection; it is not an instruction to expose unencrypted HTTP on a public server. The first-user flow creates an administrator, so complete it while only you can reach the service. [S13]

Upload a few owned sample images and a short sample video. Check that processing finishes, thumbnails appear and the originals can be downloaded. Restart the containers without deleting storage and confirm the sample library remains.

Step 6 - Learn the mobile and recovery boundaries

The laptop tunnel is not a phone endpoint. For a mobile extension, design a private HTTPS address reachable by the enrolled phone, with a certificate the actual app trusts. Test foreground upload, background operation, permissions and reconnection separately. A browser login does not prove all of those work.

Do **not** enable automatic local deletion in this exercise. Keep the originals until a verified restore from an independent backup has succeeded. “Uploaded”, “synchronised”, “in the recycle bin” and “recoverable after losing the server” are different states.

Use the selected release's backup and restore instructions for its database, and back up the corresponding media tree. Test recovery on a separate instance with compatible versions. A storage quota or a second directory on the same physical disk does not protect against loss of that disk. [S15]

Finish with: matching release files, recorded digests, loopback-only binding, an initial admin, verified sample upload/download, persistence and a documented restore plan. The mobile extension is not complete until tested on the phone.

References: S13-S15, S22; original context P01.

07 | Security monitoring, private logs and DNS filtering

Why this is a security project

The original homelab used Uptime Kuma, Dozzle and AdGuard alongside its apps. The security goals are to notice failures, preserve useful evidence and control where DNS queries go without exposing an administration dashboard or sensitive logs. Monitoring is not equivalent to intrusion detection, and a green HTTP check does not prove that permissions, encryption or backups are correct. [P01]

This chapter uses the actual Uptime Kuma instance in the core stack. There is no separate demonstration API or autonomous agent. DNS filtering and Dozzle are scoped extensions to the existing-service work, not copied production configurations.

Step 1 - Protect monitoring access

Complete chapters 01-03. Open `https://status.lab.test:8443` through the SSH tunnel, create a fresh administrator and keep this interface private. Record the selected Kuma release. Do not put actual service names, failure messages or infrastructure addresses on a publicly accessible status page. [S17]

The core Compose file gives Kuma persistent storage and a network shared with Caddy. It deliberately does not mount the Docker socket. A read-only filesystem mount of that socket does not make the Docker API read-only; access to a rootful daemon remains highly privileged. [S02]

Step 2 - Add a clearly scoped check

Append this lab-only internal endpoint to `labs/core/Caddyfile`:

```
http://:2015 {  
    respond /health-demo "classlab-ok" 200  
}
```

Do not add port 2015 to the Compose host-port list. From `labs/core` on the server, recreate Caddy after checking its configuration:

```
sudo ./lab.sh exec caddy caddy validate \  
    --config /etc/caddy/Caddyfile --adapter caddyfile  
sudo ./lab.sh up -d --force-recreate caddy
```

In Kuma, create an HTTP keyword monitor for `http://caddy:2015/health-demo`, requiring `classlab-ok`. Use a 60-second interval for this exercise. The check is on the container network; it does not verify the laptop's DNS, TLS trust, SSH tunnel or application login. State that limitation in the evidence. [S17]

Step 3 - Produce and recover from an actual failure

Change only the endpoint response text to `classlab-maintenance`, recreate Caddy and observe the monitor after its next checks. A keyword failure should appear even though the HTTP service responds. Restore `classlab-ok` and record the recovery time. Do not invent a response time or uptime percentage.

An optional notification uses an account/channel you control. Keep webhook URLs, tokens and recipient identifiers outside Git. Confirm delivery with this small failure rather than testing against a real outage or posting a private service address into a class channel.

Step 4 - Inspect relevant logs without redistributing them

On the lab server, these commands read existing logs:

```
sudo journalctl -u ssh --since "30 minutes ago" --no-pager
cd ~/class-project-pack/labs/core
sudo ./lab.sh logs --since 30m --tail 100 caddy vaultwarden
```

Make one intentional failed login to a disposable account you own, then look for the corresponding event. Do not brute-force passwords. Note timestamps and whether the logged client identity represents the real client or a proxy. Remove addresses, usernames, tokens and file names before sharing excerpts. Do not switch an app to verbose request/body logging just to get a screenshot.

Dozzle can provide a convenient private view of container logs, but its socket access is a separate trust decision. The baseline here uses CLI logs instead of adding a new privileged integration. Retain a working private Dozzle setup only after reviewing its own authentication and Docker API boundary. [S02, S30]

Step 5 - Reproduce an AdGuard DNS-policy test

On a separate disposable host or VM, follow AdGuard Home's current installation and first-run guide, S31. Configure an administrator and limit the DNS listener to your intended LAN or private-overlay address. Permit TCP/UDP 53 only from authorised clients. Keep the setup/admin UI private; remove temporary setup exposure after enrolment. Never publish an unrestricted recursive resolver.

Do not change the whole household's DHCP or DNS while learning. Point a single query at your new resolver first. On a Linux test client with `dnsutils`:

```
dig @YOUR_LAB_DNS_IP example.com A
dig @YOUR_LAB_DNS_IP example.com AAAA
```

In AdGuard's custom rules, temporarily add `||example.com^`. Repeat the explicit query and inspect the query log. Record whether the selected blocking mode returns a blocking address, an empty result or a DNS error. Remove the rule and confirm normal resolution returns. The rule is a harmless policy test, not a claim that the example domain is malicious. [S31]

Understand DNS limits before changing clients

A configured resolver is not proof every app uses it. Browser DNS-over-HTTPS, Android Private DNS and VPN-provided resolvers can use different paths. Verify one controlled device before changing system-wide settings. Private names need an appropriate resolver path, not just network reachability. The original work included cases where private DNS/mobile resolution still needed verification. Do not present that unfinished testing as a completed universal fix. [P01]

Blocking a domain is not a firewall rule for all traffic to its IP. DNS filtering does not inspect encrypted file contents or prove a download safe. Retain only the query history you need; DNS logs themselves reveal activity.

Evidence and completion

Provide the monitor's defined scope, observed failure/recovery, one reviewed log event and the DNS-rule test outcome. Add a denied attempt from an unauthorised client to the monitoring/admin interface. A complete result explains what the controls do **not** measure as well as what passed.

References: S02, S17, S30-S31; original context P01.

08 | Harden the host and container boundary

Original hardening and the classroom target

The original work reported SSH restricted to NetBird, closed Vaultwarden registration, a disabled Vaultwarden server-admin panel, two-factor setup, HTTPS access and careful separation of selected public apps from private administration. Nextcloud work included reverse-proxy trust, Redis locking, scheduled jobs and storage limits. [P01]

This chapter turns those goals into a checkable baseline. The exact SSH snippet and narrow firewall guard supplied here are new examples, not exports of the original host. Changing authentication or firewall rules can lock you out: use your own disposable server, retain a provider/local console and keep a known-good SSH session open until a **new** session passes.

Step 1 - Inventory the current exposure

Run on the lab server and keep the full output private:

```
sudo ss -ltnp
sudo systemctl --failed
sudo systemctl is-enabled ssh
sudo docker ps --format 'table {{.Names}}\t{{.Ports}}'
```

For each listener, record its purpose, bound address, required source clients and authentication. A listener on `127.0.0.1` is different from `0.0.0.0` or `:::`. A process being installed is different from it listening publicly. Do not disable an unfamiliar service until you understand its dependencies.

The core web stack publishes only loopback TCP 8443. PostgreSQL and Redis have no host-published ports. Docker network membership gives connected containers reachability; it is not per-request authorisation. A compromised application can still reach its own database. [S02, S24]

Step 2 - Establish key-based SSH before removing passwords

On the laptop, create a dedicated key with a passphrase:

```
ssh-keygen -t ed25519 -a 64 -f ~/.ssh/classlab_ed25519
```

If that path already exists, choose a new filename rather than overwriting a key. Transfer only its `.pub` contents to the lab user's `~/.ssh/authorized_keys` using the existing authorised connection or console. On Linux, `ssh-copy-id` can perform this enrolment:

```
ssh-copy-id -i ~/.ssh/classlab_ed25519.pub \
  labuser@YOUR_LAB_NETBIRD_IP
ssh -o IdentitiesOnly=yes -i ~/.ssh/classlab_ed25519 \
  labuser@YOUR_LAB_NETBIRD_IP
```

Replace `labuser` with the ordinary account created for the lab. Verify the server's host-key fingerprint against its trusted console, and confirm `sudo` works in this new session. Windows OpenSSH has `ssh-keygen` and `ssh`, but not normally `ssh-copy-id`; install the public key via the server console instead. Never copy the private key to the server or class repository.

Step 3 - Apply a small SSH baseline

Inspect the existing configuration and `/etc/ssh/sshd_config.d/` first. OpenSSH usually uses the first obtained value for a setting, and `Match` rules can alter effective settings. A new file alone does not prove the desired values win. This is why the effective-configuration test below matters. [S25]

The supplied `labs/hardening/00-classlab.conf.example` contains:

```
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
PermitEmptyPasswords no
X11Forwarding no
AllowAgentForwarding no
AllowTcpForwarding local
GatewayPorts no
PermitTunnel no
MaxAuthTries 3
LoginGraceTime 30
```

Local TCP forwarding is deliberately retained: the HTTPS exercise depends on it. This is not a restricted-shell policy; an account with shell or Docker/sudo privileges can still run other networking tools. Do not use this baseline on a server that requires keyboard-interactive MFA without redesigning that login flow first. [S25]

From the repository root, **only after key login works**:

```
sudo install -m 0644 labs/hardening/00-classlab.conf.example \
/etc/ssh/sshd_config.d/00-classlab.conf
sudo /usr/sbin/sshd -t
sudo /usr/sbin/sshd -T | grep -E \
'permitrootlogin|passwordauthentication|kbdinteractiveauthentication|allowtcpforwarding|gatewayports'
```

When syntax or effective values are wrong, fix them before reloading. Review applicable `Match` sections with `sshd -T -C` for the real lab user/source as needed. On Debian's ordinary SSH service:

```
sudo systemctl reload ssh
```

Open another key-authenticated session and a fresh HTTPS forward. Then test a password-only attempt with an owned account; it should not offer a successful password login. Keep the original connection/console until both results are known. Remove the new snippet and reload from that session if rollback is needed.

Step 4 - Make administration NetBird-only

NetBird policy must allow only the intended administrator group to reach the lab server's TCP 22. Remove an overlapping broad default grant after testing the narrow rule. Application credentials still apply. Then independently deny SSH arriving outside the overlay at the host boundary. [S05]

The example `labs/hardening/ssh-guard.nft.example` is a **narrow deny guard**, not a complete host firewall. Replace `YOUR_NETBIRD_INTERFACE` with the verified interface name from your own route/link inspection. It rejects TCP 22 on any other interface, for both IPv4 and IPv6; it does not flush Docker/NetBird rules or change every other service's policy. Existing restrictive rules can still block SSH, so confirm the complete policy privately. [S24, S29]

After filling the template into a private file outside Git:

```
sudo nft -c -f /etc/classlab/ssh-guard.nft
sudo nft -f /etc/classlab/ssh-guard.nft
sudo nft list table inet classlab_ssh_guard
```

The first command checks syntax; it does not prove packet behaviour. Keep console access while applying the second command. Do not enable or replace a generic `nftables.service` that flushes a host's existing rules without reviewing it. The supplied dedicated unit loads only this guard at boot. Follow its README, verify a reboot with console access, and never assume an in-memory rule persists.

A host loopback SSH attempt is also denied by this example. This is intentional: administration arrives through NetBird. This guard does not block the **destination** of an established SSH local-forward channel at loopback 8443.

Step 5 - Check container privileges and published ports

From `labs/core`, validate bindings without printing environment secrets:

```
sudo ./lab.sh config --format json | \
python3 ../../scripts/check_bindings.py
```

The pipe carries expanded configuration locally; do not redirect it to a public file or enable shell tracing. The helper prints only port bindings and rejects unspecified/public addresses or host networking in the core stack.

Docker's published ports are not reliably controlled by a simple UFW rule in the way a host-only listener is. Keep unnecessary ports unpublished, understand the active Docker firewall backend and test from a second machine. Do not replace Docker's generated rules with an unreviewed `flush ruleset`. [S24]

The core stack avoids host networking, privileged containers and socket mounts. Its configuration files are read-only mounts where practical; its application data remains writable. Adding `read_only`, dropping all capabilities or forcing a random UID can break an image's initialisation. Test such changes separately against that image rather than claiming a universal hardening flag. [S02]

Step 6 - Harden accounts, recovery and maintenance

For Vaultwarden, close registration after the deliberate first-user exercise, keep the separate server-admin endpoint disabled and enrol two-factor verification. Store recovery material outside the vault. For Nextcloud, use an ordinary daily account, narrow proxy trust and a tested quota. A quota is not a physical reservation or a backup. Chapters 04-06 provide those tests.

Keep local `.env`, transport profiles and backup passwords out of Git; use restrictive file permissions. `docker inspect` and resolved Compose output can reveal environment secrets. A secret scanner is not a substitute for access control or a manual review.

Before upgrades, record current versions/digests, read migrations, make a backup and test recovery. Recheck exposure and access after upgrades and reboot. Do not enable uncontrolled database-major updates on a working data directory. The original hardening is not a guarantee that future configuration changes remain safe. [S03, S12, S15]

Completion evidence

Record: new key login succeeds; password-only login fails; an authorised overlay peer reaches TCP 22; a LAN/unpermitted peer does not; no database is published; the core binding check passes; the HTTPS tunnel still works; and the guard survives a controlled reboot. Treat every unrun case as **not tested**.

References: S02-S03, S05, S12, S15, S24-S25, S29; original context P01.

09 | Build a custom Xray/REALITY privacy proxy

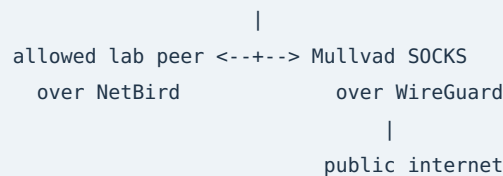
Original project and security objective

The original custom proxy used Xray/REALITY with selective routing: ordinary internet traffic used Mullvad egress while private service destinations retained a NetBird path. NetBird management and the proxy had separate VPS roles. Later app work was required to preserve the existing proxy rather than repurpose its ports or route policy. [P01]

This is a new, smaller classroom implementation of that design. It supplies client/server templates and failure tests, not the author's transport keys, subscription link or production routing table. It begins with **explicit TCP application proxying**, not a system-wide VPN. Only apps using the supplied local SOCKS port participate. Other applications remain outside this test.

Architecture and prerequisites

Test app -> laptop SOCKS -> VLESS/REALITY -> separate proxy VPS



Use a fresh Debian proxy VM with console access, its own public IPv4, an owned DNS hostname and permission to run the service. A self-hosted NetBird controller is a separate role from chapter 02. You also need your own valid Mullvad account and a disposable NetBird lab peer. These external resources can incur costs; no account or infrastructure belonging to the author is supplied.

On the proxy VM, complete the Linux/Docker foundation and enrol it in your own NetBird mesh. Preserve private administrator SSH. For the owned TLS target below, TCP 80/443 are deliberately public; the authenticated proxy listens on TCP 8443. Restrict 8443 to your test client's public source IP at the provider firewall for this lab where practical. Do not open home database or administration ports.

The TLS target is a harmless certificate/handshake endpoint, not an application or content site. Keep it separate from password vaults, files and admin pages.

Step 1 - Write the outbound policy before deploying

Request	Intended handling	Failure expectation
One owned peer IPv4, TCP 8088	Private NetBird path	No fallback onto the physical interface
Other private/reserved IP targets	Deny	No broad access to the mesh or VPS LAN
Ordinary public TCP target	Mullvad SOCKS through WireGuard	Request fails when that egress is absent
UDP or literal IPv6 target	Deny in this baseline	Not claimed as supported/tested
Private-name suffixes such as <code>lab.test</code>	Deny in this baseline	Do not leak those names to public egress

Xray evaluates rules in order and defaults to the first outbound if none matches. The template therefore makes the provider SOCKS path the first outbound, places a single narrow private allow before the private-address deny and has no public `freedom` fallback. Its only direct outbound is reachable through the specific permitted peer/port rule. [S18]

The denied literal-IPv6 rule is not a system-wide IPv6 kill switch. Public hostnames are forwarded for resolution by the provider proxy, which may choose an IPv4 or IPv6 destination. That connection remains on the provider side of the encrypted path. The laptop's unrelated IPv6 traffic is outside this lab.

Step 2 - Prepare the files in a private workspace

On the proxy VM, copy the repository there through your own authorised SSH connection. Then create an ignored workspace and copy the templates:

```
cd ~/class-project-pack/labs/proxy
mkdir -m 700 runtime
cp server.json.example runtime/server.json
cp wg-lab.conf.example runtime/wg-lab.conf
cp proxy-guard.nft.example runtime/proxy-guard.nft
cp Target.Caddyfile.example runtime/Target.Caddyfile
cp target.compose.yaml.example runtime/target.compose.yaml
chmod 600 runtime/*
```

If the workspace exists, inspect it instead of overwriting working profiles. All `REPLACE_...` values must be filled privately. Keep client configuration in a similarly private directory **on the laptop**. It must never contain the server's private key or Mullvad's WireGuard private key.

Step 3 - Establish an owned TLS target

Create a DNS A record for an owned hostname such as the illustrative `tls.example.com`, pointing to the proxy VM. Replace `REPLACE_TLS_HOSTNAME` in the target Caddyfile. Do not create an AAAA record unless the intended IPv6 listener/firewall path has been configured and tested. The example name itself cannot be used to obtain your certificate.

On the proxy VM, from the `runtime` directory:

```
cd ~/class-project-pack/labs/proxy/runtime
sudo docker compose -f target.compose.yaml config -q
sudo docker compose -f target.compose.yaml \
  config --lock-image-digests -o target.images.yaml
sudo docker compose -f target.compose.yaml \
  -f target.images.yaml up -d
```

This separate Compose file intentionally publishes 80/443. It must not be merged into the private application stack. Caddy obtains a certificate for the owned name when DNS and validation are correct. Its internal data directory contains private CA/account material and stays out of Git. [S07]

Check the local TLS target using the real owned hostname in place of the placeholder. Require a valid certificate, TLS 1.3 and negotiated `h2`:

```
openssl s_client -connect 127.0.0.1:443 \
  -servername YOUR_OWNED_TLS_HOSTNAME -tls1_3 -alpn h2 \
  -verify_hostname YOUR_OWNED_TLS_HOSTNAME -verify_return_error
```

Do not continue while validation fails. The Xray server uses local target `127.0.0.1:443` and the matching hostname. REALITY can send unauthenticated handshakes to its configured target, which is why this lab uses an owned, harmless endpoint rather than forwarding strangers into a sensitive app. [S26]

Step 4 - Create selective Mullvad egress

Return to the template directory on the proxy VM and install the required tools:

```
cd ~/class-project-pack/labs/proxy
sudo apt install wireguard-tools nftables curl openssl unzip
```

Commands from this point use that directory unless a different machine is named. Generate a fresh WireGuard profile through your own Mullvad account. From that profile, privately copy the assigned IPv4/CIDR, client private key, provider peer public key and numeric endpoint/port into `runtime/wg-lab.conf`. Do not copy the provider's full default-route profile over this selective lab.

Retain only `AllowedIPs = 10.64.0.1/32` from the template. Do not add global DNS, `0.0.0.0/0`, `::/0` or full-tunnel firewall hooks. `wg-quick` will create the specific route; it is not intended to replace the VPS's default route or its SSH return path. The address `10.64.0.1:1080` is the documented Mullvad SOCKS endpoint, not an address from the author's infrastructure. [S32-S33]

Install the filled profile and start its lab-specific service:

```
sudo install -m 0600 runtime/wg-lab.conf /etc/wireguard/wg-lab.conf
sudo systemctl enable --now wg-quick@wg-lab
ip route get 10.64.0.1
sudo wg show wg-lab
```

Confirm the route names `wg-lab`, generate a request below and inspect the handshake/counters. Keep key, peer and endpoint output private. If the profile cannot reach this SOCKS service, stop and compare it with current provider instructions rather than enabling a broad default route as a shortcut.

Step 5 - Add a narrow egress guard before client traffic

In `runtime/proxy-guard.nft`, replace the lab peer IPv4 and actual NetBird interface. The first rule rejects provider-SOCKS traffic not leaving `wg-lab`; the second rejects the allowed lab-peer traffic not leaving the overlay. Neither rule replaces the host's complete firewall. [S29]

```
sudo install -d -m 0750 /etc/classlab
sudo install -m 0600 runtime/proxy-guard.nft /etc/classlab/proxy-guard.nft
sudo nft -c -f /etc/classlab/proxy-guard.nft
sudo install -m 0644 classlab-proxy-guard.service \
  /etc/systemd/system/classlab-proxy-guard.service
sudo systemctl daemon-reload
sudo systemctl enable --now classlab-proxy-guard
sudo nft list table inet classlab_proxy_guard
```

Load the table once. Read `labs/proxy/README.md` before reloading or applying a file already loaded manually. Never use `nft flush ruleset`. If a route vanishes, the guard is intended to stop the same destination escaping through another interface. The interruption test still has to prove the actual behaviour.

From the proxy VM, verify the provider path without changing the system proxy:

```
curl --noproxy '' --max-time 15 \
  --socks5-hostname 10.64.0.1:1080 \
  https://am.i.mullvad.net/connected
```

The response should identify Mullvad egress. Keep the reported address private. `--socks5-hostname` delegates hostname resolution for this request instead of resolving the requested website locally. SOCKS itself is not encryption; this provider hop is protected by WireGuard. [S28, S32]

Step 6 - Install a compatible Xray core and generate identities

Use the official Xray-core release page, S23. On an x86-64 Debian lab, download the selected release's `Xray-linux-64.zip` and matching `.dgst` checksum file. For other CPU architectures choose the corresponding official asset instead. Do not execute an unreviewed third-party install script.

```
uname -m
# Replace with the selected official release tag, including its leading v.
XRAY_VERSION='REPLACE_WITH_COMPATIBLE_RELEASE_TAG'
curl -fL "https://github.com/XTLS/Xray-core/releases/download/${XRAY_VERSION}/Xray-linux-64.zip" \
  -o runtime/Xray-linux-64.zip
curl -fL "https://github.com/XTLS/Xray-core/releases/download/${XRAY_VERSION}/Xray-linux-64.zip.dgst" \
  -o runtime/Xray-linux-64.zip.dgst
sha256sum runtime/Xray-linux-64.zip
cat runtime/Xray-linux-64.zip.dgst
```

Compare the SHA-256 value with the release checksum before extraction. Record the exact version and hash. This identifies the artifact; a checksum from the same source is not independent proof of the publisher's trustworthiness. Extract only after the values agree, then install the binary under a lab name:

```
unzip runtime/Xray-linux-64.zip -d runtime/xray-release
sudo install -m 0755 runtime/xray-release/xray /usr/local/bin/xray-lab
xray-lab version
xray-lab uuid
xray-lab x25519
openssl rand -hex 8
```

Generate a fresh UUID, REALITY key pair and short ID. Copy them directly into private files; do not post the output. The server gets the REALITY private key. The client gets the corresponding Password/public-key-derived value, which must also be kept private as profile material. Both get the same UUID and short ID. The WireGuard key pair remains separate. [S26-S27]

The templates use the current documented JSON layout (`users`, flattened outbounds, `method: raw`, client `password`). An older bundled GUI core may use older field names. Select compatible client/server cores and require their own configuration tests to pass; structural JSON parsing alone is insufficient.

Step 7 - Install and test the server profile

Fill the server JSON with the generated identity fields, owned TLS hostname and one lab-peer IPv4. The direct allow is restricted to that peer's TCP 8088. Check the policy with the supplied helper, then use Xray's own parser:

```
python3 ../../scripts/check_proxy.py runtime/server.json --ready
xray-lab run -test -config runtime/server.json
```

The helper prints only a verdict, never the credential values. It checks a small intended policy, not every possible Xray option or security property. Do not start the listener until both checks succeed.

Create the dedicated service account only if it does not already exist:

```
id xraylab || sudo useradd --system --no-create-home \
  --shell /usr/sbin/nologin xraylab
sudo chown root:xraylab /etc/classlab
sudo chmod 0750 /etc/classlab
sudo install -o root -g xraylab -m 0640 runtime/server.json \
  /etc/classlab/xray-server.json
sudo install -m 0644 xray-lab.service /etc/systemd/system/xray-lab.service
sudo systemctl daemon-reload
sudo systemctl enable --now xray-lab
sudo systemctl status xray-lab --no-pager
```

The service runs without root, depends on the guard loader and has no writable host filesystem requirement. Guard loading failure should prevent startup. These are proposed classroom controls, not an executed authoring result. Read errors privately and fix the failed requirement rather than running the whole proxy permanently as root. [S35]

Step 8 - Start an explicit client, then verify egress

On the laptop, install the same compatible official core for its platform. Copy `client.json.example` to a private `client.json`, restrict file access, and fill the server's public IPv4, UUID, owned TLS hostname, client REALITY password and short ID. No server-side private key belongs on the laptop.

The laptop SOCKS listener binds to `127.0.0.1:10808`, with UDP disabled. It has only one outbound and no configured direct fallback. Run the core in the foreground so it can be stopped with Ctrl+C:

```
xray run -test -config client.json
xray run -config client.json
```

Use `xray-lab` on Linux if that is the binary name you installed, or `xray.exe` on Windows. In another laptop terminal:

```
curl --noproxy '' --max-time 15 \
  --socks5-hostname 127.0.0.1:10808 \
  https://am.i.mullvad.net/connected
```

On PowerShell use `curl.exe`, and write the command on one line instead of copying Bash backslash continuations. The reported egress should be Mullvad, not the laptop's ordinary connection or the proxy VM's public address. This establishes only this request's path, not a whole-device privacy audit.

Step 9 - Prove the private NetBird branch

On the disposable homelab peer, create one harmless marker outside real data and bind a temporary server only to its overlay IPv4:

```
mkdir -m 700 ~/classlab-proxy-check
printf 'netbird-path-ok\n' > ~/classlab-proxy-check/health.txt
python3 -m http.server 8088 --bind YOUR_LAB_PEER_IPV4 \
  --directory ~/classlab-proxy-check
```

Allow only the proxy VM's peer group to reach this peer on TCP 8088 in NetBird. No router port forward is needed. The Python server is a temporary diagnostic endpoint, not a production service; stop it after the test. From the laptop:

```
curl --noproxy '' --max-time 10 \
  --socks5-hostname 127.0.0.1:10808 \
  http://YOUR_LAB_PEER_IPV4:8088/health.txt
```

Expect the marker. An owned negative test to the same peer's TCP 22 through this proxy should fail because the Xray policy permits only 8088. A separate NetBird administrator session may still use SSH; that is a different path. The server sees the proxy peer as the network source, so every authorised proxy client shares that network identity. Do not treat this as per-user NetBird authorisation or open broad mesh access to untrusted clients.

Step 10 - Test interruption, recovery and limits

Keep the guard loaded and console/private management available. With only the harmless requests running, test these one at a time:

Test	Expected result
Stop <code>wg-quick@wg-lab</code> on the proxy VM	New public proxy requests fail; no ordinary-internet fallback
Restart WireGuard	Public requests recover through Mullvad
Disconnect the test service peer from NetBird	Private-marker request fails; public proxy requests still work
Restore that peer	Private request recovers
Stop laptop Xray	Explicit SOCKS requests fail rather than use a system route
Use a wrong client UUID in a separate private test profile	Cannot obtain proxy access
Reboot proxy VM with console access	Guard, profiles and intended routes recover before approval

Do not disconnect the only remote-management path to perform a test. Inspect fresh requests, not just a GUI status label. Record the actual results and untested cases. A failure is a useful finding, not something to hide. Provider rate limits, source-IP firewall restrictions and TLS-target errors can also cause a connection to fail; distinguish those causes.

The original Windows/Android deployment used client routing/TUN features too. They are an advanced extension here: verify bootstrap exclusions, DNS, UDP, IPv6, private-name resolution, suspend/resume and reconnect behaviour separately before sending all device traffic through it. Do not replace a working desktop routing setup with this lab profile. This pack does not claim a validated full-device kill switch or invisible/untraceable traffic.

References: S18, S23, S26-S29, S32-S36; original context P01.

10 | Separate public applications from private management

What was done in the original environment

The original deployment deliberately made selected apps reachable remotely without requiring every client to join the management network. Vaultwarden and Nextcloud used Cloudflare Tunnel. The photo service used a separate, scoped VPS HTTPS edge with a private path to its origin. Administration remained private, and later ingress work was required to preserve the existing custom proxy. These are reported outcomes, not a new live audit. [P01]

The central lesson is **expose only the application path you intend**, while retaining application authentication and independently restricting the origin. A working public login page does not imply public SSH should also work. A private management network does not automatically make every Docker listener private.

Step 1 - Define the intended reachability

Use a separate policy table for your lab. The default classroom stack is narrower than the original selected-public-app design:

Surface	Classroom default	Evidence required
SSH	Approved administrator peer over NetBird	New authorised login works; other sources denied
Vault/file/photo apps	Private tunnel or independent private photo lab	Intended login works; unauthorised data access fails
PostgreSQL / Redis	Container backend network only	No host-published database port
Kuma / log viewer / DNS administration	Private only	No external administrative access
Custom proxy on separate VM	Authenticated TCP 8443, limited test sources	Valid profile works; wrong credentials fail
Owned proxy TLS target	Public, harmless response on 80/443	Contains no private app or admin route

Do not apply the core loopback-only port checker to the deliberately public TLS-target stack and call its expected failure a vulnerability. Each stack has a different documented exposure policy. Test the appropriate policy.

Step 2 - Verify the private HTTPS baseline

From the laptop with the chapter 03 tunnel and verified CA certificate:

```
curl --cacert classlab-root.crt \
  --resolve check.lab.test:8443:127.0.0.1 \
  https://check.lab.test:8443/healthz
```

Expect the fixed `classlab-https-ok` marker. Close the SSH tunnel and repeat: the request should no longer reach the remote lab. From another owned LAN client, test the server's LAN address on TCP 8443; it should not expose this site. Confirm the test client really has a path to the server before attributing all failures to your security controls. [S07, S24]

Test authentication separately. Use an incognito/new lab profile with no login cookie and confirm private files/vault contents are not available. A 200 login page is not a failed security test; an unauthenticated response containing private data would be. Test only owned demo accounts and harmless files.

Step 3 - Understand Tunnel, Access and origin authentication

Cloudflare Tunnel provides a connector path from an origin to Cloudflare; Cloudflare Access is a separate access-policy layer. A published Tunnel hostname is not made user-private merely by having a tunnel. The app may still be intentionally reachable by anyone who can attempt its own login. The original Vaultwarden/Nextcloud work should not be relabelled as Access-protected without evidence of such a policy. [S16, S34; P01]

An interactive browser Access login can interfere with a native app expecting an API response. Machine access may require a service token and compatible headers; not every client supports that. The original RSS/mobile work exposed this distinction. Do not solve a failed mobile API test by broadly removing authentication or bypassing all API paths. Choose a deliberately private path or a documented, client-compatible authentication design. [S16]

Even a correctly authenticated edge should not allow direct public access to an origin that bypasses the edge. Identify every listener, router forward, Docker published port, DNS record and alternate hostname before concluding that the application is reachable through only the intended path.

Step 4 - Design a controlled public-ingress extension

This is an advanced design/verification extension, not an automatic publication of the core apps. First prove the private stack works. Start public tests with only the harmless health endpoint and an owned domain; keep real accounts and files out of the experiment.

For an independent Cloudflare Tunnel, use S34's current connector and ingress instructions in a private workspace. Match explicit hostnames, end with a non-routing/catch-all rejection, and keep generated credentials out of Git. If the connector uses HTTPS to your origin, verify its certificate and name; do not disable origin certificate verification as a permanent fix.

Before placing a real self-hosted app behind a different external hostname, update its documented external URL and trusted-domain/proxy settings. For Nextcloud, retain a narrow trusted-proxy list and verify DAV behaviour. A successful health route does not prove the application's login, uploads, WebSocket connections or mobile API will work. [S10-S11, S34]

For a VPS reverse-proxy edge, restrict the origin route to the exact peer and app port. The proxy's network identity should not gain blanket homelab access. Keep the custom Xray listener and configuration untouched; list occupied ports and confirm a rollback plan before adding an edge listener. Re-run both public egress and private-routing tests afterwards. [S05; P01]

Step 5 - Collect positive and negative evidence

On your own Linux test machines, a basic single-port test can use Python:

```
python3 -c "import socket; s=socket.create_connection(('YOUR_LAB_IP',8443),3); s.close(); print('TCP connected')"
```

Replace the placeholder with one owned endpoint. Run it from the approved source and a separate unapproved source. A connection exception is an observed connection failure, not proof which firewall caused it. Combine it with your listener and rule inspection; do not scan unrelated networks.

For the selected app, record logged-out behaviour, correct login, a wrong password attempt, another normal user's denied data access, and the expected HTTP/API result. Check the real external hostname and the direct-origin path independently. Testing one browser over Wi-Fi does not cover mobile data, IPv6, a different DNS resolver or a second ingress route.

Step 6 - Preserve privacy when sharing the results

Share a table of expected versus observed outcomes using role labels such as `admin laptop`, `unapproved peer` and `file service`. Reconstruct topology with fictional addresses. Do not publish live DNS inventories, NetBird peer IDs, account email addresses, authentication headers or complete reverse-proxy logs.

Removing a name from a report protects the report's privacy; it does not enforce access control on the service. Keep access restrictions effective even when an attacker knows the hostname. Add a limitation whenever a test did not establish which layer caused an outcome.

Completion evidence

Finish with a reviewed exposure table, at least one permitted and one denied network test, one authentication/authorisation test and a no-regression check for the custom proxy. Mark external-ingress or native-client steps **not tested** when they were only designed. Do not claim the class pack audited the original production network.

References: S05, S07, S10-S11, S16, S24, S34; original context P01.

11 | Encrypted backups and a real restore drill

Original lesson: storage budgets are not recovery

The original file/photo work used separate allocations and planned backup space. That is useful capacity management, but a second directory on the same physical disk is not protection against losing that disk. A quota, mirror, recycle bin, synchronised client and recoverable backup solve different problems.

This classroom exercise stops the small core stack and uses restic to make an encrypted backup. It is a deliberately simple cold-backup method, accepting brief downtime instead of pretending that a live file copy guarantees database consistency. It covers `labs/core`, **not the separate Immich or NetBird stacks**. Use their own matching-version recovery instructions for those services.

Step 1 - Identify the complete recovery set

The core directory contains the application files, PostgreSQL data, Vaultwarden state, Kuma state, Caddy's private CA material, configuration, image lock, site files and `.env`. All of those can be sensitive. The backup is not suitable for upload to GitHub just because the archive is compressed.

Redis is configured as a non-persistent cache for this exercise. The authoritative file and database state belongs to the applications. The cold backup preserves PostgreSQL's physical data, so restore with the recorded compatible image and major version before attempting upgrades. [S12]

Step 2 - Prepare a genuinely separate destination

Use a mounted external drive or another appropriately protected repository. For the example commands, `/mnt/backup` is an already-mounted destination. This guide does not partition or format it. Check that the intended backup filesystem is actually mounted; a missing mount can cause writes to land on the same system disk you thought you were protecting.

Install restic on the lab server and create a private password file:

```
sudo apt install restic
mountpoint /mnt/backup
sudo install -d -m 700 /root/.config/classlab
sudo sh -c 'set -C; umask 077; openssl rand -hex 32 > /root/.config/classlab/restic-pass'
```

Stop if the mount check does not identify the intended mounted destination. `set -C` prevents overwriting an existing password file: changing it blindly can make old backups inaccessible. Keep an additional secure recovery copy of the password somewhere separate from the server and backup disk. Never publish it.

Initialise a new restic repository once:

```
sudo restic -r /mnt/backup/classlab \
-p /root/.config/classlab/restic-pass init
```

The repository is encrypted using restic's supported design. Encryption does not remove the need for access control, a recoverable password and a destination that survives the relevant failure. [S19]

Step 3 - Run the stopped-stack backup

From the repository root on the lab server:

```
sudo scripts/backup_core.sh \
  /mnt/backup/classlab /root/.config/classlab/restic-pass
```

The helper verifies the repository can be opened before stopping services. It records which core services were running, stops the stack, backs up the core folder, and attempts to restart only the previously running services even when the backup fails. A failure remains a failure: the script does not label a partial or interrupted backup successful.

Read its output privately. The backup tool can show hostnames and paths. Verify that the applications restarted and that the command exit status was successful. A script saying it “started a backup” would not prove one was completed.

Step 4 - Verify and restore into a new location

List snapshots and check the repository:

```
sudo restic -r /mnt/backup/classlab \
  -p /root/.config/classlab/restic-pass snapshots
sudo restic -r /mnt/backup/classlab \
  -p /root/.config/classlab/restic-pass check
```

Choose an actual snapshot ID from your own output. On an isolated recovery VM or a new empty test location, restore it using the recorded ID:

```
SNAPSHOT_ID='REPLACE_WITH_YOUR_SNAPSHOT_ID'
sudo restic -r /mnt/backup/classlab \
  -p /root/.config/classlab/restic-pass \
  restore "$SNAPSHOT_ID" --target /srv/classlab-restore-test
```

Do not restore over the working source to “test” your only backup. The restored files appear below the target with their recorded directory structure. Locate the restored core folder, inspect it and retain its matching `.env` and image lock. On a separate VM, start only the services you need to verify recovery. Use a new SSH tunnel to that VM; do not run two stacks on the same conflicting loopback port or connect the restored service to production clients.

Step 5 - Prove application-level recovery

A successful archive extraction is not the final test. Log into the restored fake vault and read the demo item. Log into Nextcloud and download the test files. Compare their hashes with the known originals. Confirm the normal user still cannot see another user's private files. Check that the monitoring configuration and HTTPS health endpoint also work.

Record the snapshot, matching images, restore steps and actual verification. Do not record the passwords. Only after a successful drill should you describe the backup process as tested for that particular scenario.

Retention and other services

This edition does not schedule destructive retention or pruning commands. Plan retention after measuring how much data changes, how much recovery history is needed and how much space is available. Monitor failed backups and perform periodic restores; a growing directory of untested archives is weak evidence.

For Nextcloud live backups, follow its documented maintenance/database method. For Immich, back up the database plus media using the selected release's instructions. For NetBird, preserve its identity/configuration state through its documented maintenance procedure. Never call the core-only script a backup of the entire homelab. [S04, S12, S15]

References: S12, S15, S19-S20.

12 | Publish the learning, not the secrets

Why a fresh teaching repository helps

A live server directory can contain credentials in places that are easy to miss: environment files, generated JSON, database backups, shell history, terminal recordings, request headers and copied troubleshooting output. A fresh repository of examples is easier to review than a production export with a few names blacked out.

This pack was built as a new teaching tree. It contains no imported production Git history and no original screenshots. That reduces exposure, but it does not establish a general guarantee about any future files you add.

The publication checklist

Before sharing, check configuration, documentation, assets and repository history. Replace actual addresses with fictional ones consistently. Remove working credentials rather than substituting a weak shared class password. Keep `.env.example`; exclude `.env`, generated data and private recovery files. Filled Xray/WireGuard profiles, client subscription links and transport identity material also stay private. Review tokenised download links and image metadata as well as visible text.

For future PDFs, drawing a black rectangle over text may leave the original text, links or attachments in the file. Use real redaction or rebuild the public document from sanitised source. Search its extracted text and inspect its metadata and embedded links after export. For images, review the flattened public export, not only the editing canvas. Do not publish a recovery QR code, a blurred secret that remains legible, or a terminal title containing a private machine identity.

A domain name is not normally a credential, but a collection of real service names can map your infrastructure. This class pack uses fictional names because the original addresses are unnecessary for understanding or reproducing it. Do not mistake fictional names for an access-control mechanism on the live system.

Create the GitHub repository without copying production history

Review the folder locally first. Then create a new empty repository in your own GitHub account. Start private while reviewing it; change visibility deliberately only after checking the intended audience and the material. No GitHub repository was created or modified while this pack was prepared.

From the sanitised repository root:

```
git init -b main
git add README.md SECURITY.md CONTRIBUTING.md LICENSE .gitignore .gitattributes
git add SOURCES.md VALIDATION.md VERSION-RECORD.md
git add docs diagrams labs scripts tests tools pdf
git add MANIFEST.txt SHA256SUMS.txt
python3 scripts/check_publish.py
git diff --cached --stat
git diff --cached
```

The helper inspects staged blobs for a small set of credential patterns and private-looking paths. It deliberately does not print matched secret values. It is not a comprehensive secret scanner: it does not certify PDF/image contents, all token formats, every address or the full Git history. Review the diff and use additional secret-scanning controls appropriate to the repository.

Once the staged material has been reviewed:

```
git commit -m "Add sanitised classroom homelab projects"
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPOSITORY.git
git push -u origin main
```

Replace both placeholders with the new repository you own. Use your existing secure GitHub authentication method; never embed an access token in the remote URL. Do not paste the example unchanged and expect it to create an account or repository. A browser upload is also possible, but the same privacy review still applies to every file selected.

What if a secret was already committed?

Revoke or rotate it first. Deleting the visible file or adding it to `.gitignore` does not remove it from old commits, forks, clones or downloaded releases. Follow GitHub's sensitive-data-removal guidance and consider every location to which the value was copied. Do not assume that rewriting history makes an already-disclosed credential trustworthy again. [S21]

Do not commit the redaction map that links fictional names back to real ones. Keep private test logs and operational version records separate unless their contents have been reviewed for publication.

Make the repository useful to another student

Start with a clear README, prerequisites, numbered steps, expected results, troubleshooting and limitations. Keep commands in text rather than only in screenshots. State where each command runs: laptop, lab server or controller. Provide the selected image digests or release record when you have actually built and verified them. A moving `latest` tag is not an immutable build record.

For future improvements, open an issue with a sanitised error description, software versions, the failing step and a minimal reproduction. Do not attach full diagnostic archives because they are convenient. Replace exact private values without changing the technical relationship that explains the issue.

Student evidence worksheet

Use a separate copy of the worksheet for each project. Fill in actual results. Leave unperformed checks explicitly marked "not tested".

Field	Student record
Project and date	
Goal and permission/scope	
Host OS and selected release/image digests	
Data path and access-control design	
Why each major component is needed	
Positive test: command, expected result, actual result	
Negative test: command, expected result, actual result	
Persistence/recovery test	
Failure encountered and justified fix	
Limitations and untested cases	
Sensitive details removed before publication	
Sources, upstream attribution and assistance disclosure	

Explain the work in your own understanding. This pack is not a set of assessment answers and should not be represented as independently produced coursework without following the relevant course rules.

References: S03, S21; repository SECURITY.md and VALIDATION.md.

Validation record for classroom edition

1.1

Prepared 17 September 2026. This record describes checks performed on the teaching pack. It is not a security certification, penetration test or claim that the original homelab or the supplied deployments were audited here.

Checks completed

Check	Result and scope
Python unit suite	28 tests passed: 8 binding checks, 4 hardening-template checks, 11 proxy-template checks, 4 staged-publication checks and 1 secret-initialiser check
Python and Bash syntax	All supplied Python files parsed; <code>lab.sh</code> and <code>backup_core.sh</code> passed <code>bash -n</code>
Core configuration	YAML parsed; the only published core port is <code>127.0.0.1:8443:443</code> ; no privileged mode, host networking or Docker socket mounts
Proxy examples	Both JSON examples parsed; the server example passed the limited policy helper; the separate TLS-target Compose example parsed as YAML
Secret initialiser	In a temporary copy, generated distinct passwords, used mode 0600, did not print values and refused overwrite
Publication helper	Disposable Git tests rejected a staged <code>.env</code> , filled client profile and synthetic token without echoing the token; a harmless README passed
Final package review	The staged package passed its limited publication helper; obsolete project files and references were removed; Markdown links and source identifiers were reviewed
PDF review	Both PDFs were rebuilt, their extracted text, links and metadata checked, and rendered pages reviewed for layout and residual identifying content

Checks used Python 3.13 on Linux. The helper scripts target Python 3.11 or newer; other platforms and Python releases have not been exhaustively tested. Template assertions test selected properties only. They cannot establish that a daemon accepts a configuration or that packets follow its intended policy.

Not tested during preparation

Docker Engine/Compose deployments, image pulls, Caddy runtime validation and application start-up were not run. YAML parsing is **not** equivalent to `docker compose config` or a successful service deployment. Image digests and installed application versions have not been fabricated.

The Xray binary was not available for its own configuration test. REALITY handshakes, NetBird enrolment and policy enforcement, Mullvad/WireGuard egress, firewall rule loading and persistence, systemd start-up, SSH effective settings, and deliberate tunnel-failure tests were not executed here. These are explicit lab verification tasks. The template helper is not a complete Xray schema, credential, cryptographic or network-security validator.

Certificate installation, Vaultwarden/Nextcloud/Immich account workflows, AdGuard filtering, real monitoring alerts, phone uploads and encrypted backup/restore were not exercised against live services. Whole-device TUN, DNS, UDP and IPv6 behaviour have not been certified. No live infrastructure or remote repository was modified for this revision.

The original project's hardening is described as reported project history. Additional SSH, firewall and proxy examples are separate teaching adaptations, not evidence that those exact configurations were deployed on the author's systems. See chapter 00 for the distinction.

Repeat and extend the checks

From the repository root:

```
python3 -m unittest discover -s tests -v
bash -n labs/core/lab.sh scripts/backup_core.sh
python3 scripts/check_proxy.py labs/proxy/server.json.example
```

On a fresh lab, perform the real program-specific checks before starting a service: quiet Compose validation, Caddy validation, `sshd -t` and effective configuration checks, `nft -c -f`, and the selected Xray core's `run -test`. Keep expanded configuration and output containing credentials private.

Then record both allow and deny tests, interruption and recovery results, and a separate-target restore. Use `VERSION-RECORD.md` and the chapter 12 worksheet. An expected result is not an observed result; mark unperformed checks as such.

Technical sources and provenance

Prepared 17 September 2026. These are primary project documentation references, not an assertion that every upstream version remains unchanged after this date. The examples and educational explanations are newly authored. Check release-specific notes before deployment, upgrades and restoration.

S01 | Docker Engine installation on Debian

Installation, supported Debian versions and Docker/firewall caveat.

<https://docs.docker.com/engine/install/debian/>

S02 | Docker Engine security

Docker daemon privileges and attack surface.

<https://docs.docker.com/engine/security/>

S03 | Docker Compose config reference

Quiet validation, interpolation and image-digest override.

<https://docs.docker.com/reference/cli/docker/compose/config/>

S04 | NetBird self-hosting quickstart

Current controller bootstrap and infrastructure requirements.

<https://docs.netbird.io/selfhosted/selfhosted-quickstart>

S05 | NetBird groups and access policies

Explicit access grants and the broad default policy.

<https://docs.netbird.io/manage/access-control>

S06 | NetBird Linux installation

Official client installation and management URL.

<https://docs.netbird.io/get-started/install/linux>

S07 | Caddy automatic HTTPS

Local certificate authority and client trust requirements.

<https://caddyserver.com/docs/automatic-https>

S08 | Vaultwarden HTTPS guidance

Secure web-vault context and proxy use.

<https://github.com/dani-garcia/vaultwarden/wiki/Enabling-HTTPS>

S09 | Vaultwarden registration controls

Registration and invitation configuration.

<https://github.com/dani-garcia/vaultwarden/wiki/Disable-registration-of-new-users>

S10 | Nextcloud reverse-proxy configuration

Trusted proxies and external URL/protocol handling.

https://docs.nextcloud.com/server/latest/admin_manual/configuration_server/reverse_proxy_configuration.html

S11 | Nextcloud Docker image documentation

Community-maintained image, PostgreSQL/Redis, persistence and environment variables.

<https://github.com/nextcloud/docker>

S12 | Nextcloud backup documentation

Application files, configuration and database recovery set.

https://docs.nextcloud.com/server/latest/admin_manual/maintenance/backup.html

S13 | Immich Docker Compose installation

Official release files and installation flow.

<https://docs.immich.app/install/docker-compose/>

S14 | Immich requirements and quickstart

Memory, CPU and database storage requirements; also see <https://docs.immich.app/>.

<https://docs.immich.app/install/requirements/>

S15 | Immich backup and restore

Database and media must both be backed up; release compatibility matters.

<https://docs.immich.app/administration/backup-and-restore/>

S16 | Cloudflare Access service tokens

Machine authentication requires compatible client/header handling.

<https://developers.cloudflare.com/cloudflare-one/access-controls/service-credentials/service-tokens/>

S17 | Uptime Kuma project documentation

Monitoring types, Docker image and local-data requirement.

<https://github.com/louislam/uptime-kuma>

S18 | Project X routing configuration

Ordered routing rules and outbound selection.

<https://xtls.github.io/en/config/routing.html>

S19 | restic repository preparation

Encrypted repository initialisation and password handling.

https://restic.readthedocs.io/en/stable/030_preparing_a_new_repo.html

S20 | restic restore documentation

Restoring to a separate target and verifying recovery.

https://restic.readthedocs.io/en/stable/050_restore.html

S21 | GitHub removing sensitive repository data

Rotate exposed credentials; deletion does not erase old copies/history.

<https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository>

S22 | Docker Compose merge and override rules

Replacing, rather than accumulating, a service port list.

<https://docs.docker.com/reference/compose-file/merge/>

S23 | Xray-core official releases

Choose a compatible core and verify the selected artifact; no binary is bundled.

<https://github.com/XTLS/Xray-core/releases>

S24 | Docker firewall and port-publishing behaviour

<https://docs.docker.com/engine/network/packet-filtering-firewalls/>

<https://docs.docker.com/engine/network/port-publishing/>

S25 | OpenSSH daemon configuration reference

Authentication, forwarding and effective settings; consult the installed manual too.

https://man.openbsd.org/sshd_config

S26 | Xray transport and REALITY configuration

<https://xtls.github.io/en/config/transport.html>

<https://xtls.github.io/en/config/transport/realty.html>

S27 | Xray VLESS configuration

<https://xtls.github.io/en/config/inbounds/vless.html>

<https://xtls.github.io/en/config/outbounds/vless.html>

S28 | Xray SOCKS outbound

<https://xtls.github.io/en/config/outbounds/socks.html>

S29 | nftables packet/interface matching

https://wiki.nftables.org/wiki-nftables/index.php/Matching_packet_metadata_information

S30 | Dozzle authentication and security considerations

<https://dozzle.dev/guide/authentication>

S31 | AdGuard Home installation and configuration

<https://adguard-dns.io/kb/adguard-home/getting-started/>

S32 | Mullvad WireGuard and provider SOCKS

<https://mullvad.net/en/help/easy-wireguard-mullvad-setup-linux>

<https://mullvad.net/en/help/socks5-proxy>

S33 | WireGuard wg-quick reference

<https://man7.org/linux/man-pages/man8/wg-quick.8.html>

S34 | Cloudflare Tunnel configuration and ingress

<https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/do-more-with-tunnels/local-management/configuration-file/>

S35 | systemd execution and privilege controls

<https://manpages.debian.org/trixie/systemd/systemd.exec.5.en.html>

S36 | Xray SOCKS inbound

<https://xtls.github.io/en/config/inbounds/socks.html>

P01 | Author-supplied project history

Sanitised summaries of the author's reported projects and outcomes. These establish project context, not an independent live-server audit. Actual domains, addresses, secrets and private media were not reproduced.

Third-party rights

Applications referenced here retain their upstream licenses. The included MIT license covers only newly authored teaching material and example code. It does not grant rights to redistribute media, commercial assets, credentials or an upstream product's proprietary components.